

Volume 06 Specifications - Model Provider Plugin Connector Specifications

Version v30 draft

README.md

Model Provider Plugin Connector Specifications

Version: v30 draft

Status: Draft

Document ID: MAL-V06-V30-README

Purpose

This release folder contains the official v30 documentation set for Model Provider Plugin Connector Specifications.

Scope

- Covers Model Ecosystem.
- Covers Provider Ecosystem.
- Covers Plugin Runtime.
- Covers Connector Runtime.

Acceptance

- All documents contain implementation-level guidance.
- The release PDF and ZIP are generated and verified.
- MANIFEST and RELEASE_NOTES are updated for v30.

Model Provider Plugin Connector Specifications Index

- [spec-06-30-01-model-ecosystem.md](#) - Model Ecosystem
- [spec-06-30-02-provider-ecosystem.md](#) - Provider Ecosystem
- [spec-06-30-03-plugin-runtime.md](#) - Plugin Runtime
- [spec-06-30-04-connector-runtime.md](#) - Connector Runtime

Model Ecosystem Specification

Title: Model Ecosystem Specification

Version: v30 draft

Status: Draft

Specification ID: MAL-V06-MODEL-ECOSYSTEM-SPEC-01

Related Blueprint Documents

- Volume 03 Master Blueprint
- Volume 04 Canonical Dictionary
- Volume 05 Engineering Standards
- Prior Volume 06 Specifications

Purpose

Define implementation-level requirements for Model Ecosystem in Mariam AI Enterprise OS so engineering teams can build, test, operate, and govern it consistently.

Scope

- Covers system responsibilities, runtime behavior, interfaces, events, storage, security, observability, and acceptance evidence.
- Includes interactions with Enterprise Core, AI Society, Runtime Ecosystem, Knowledge System, Capability System, Governance, and Infrastructure.
- Excludes application-specific code, vendor credentials, and temporary implementation shortcuts.

Responsibilities

- Maintain authoritative state and lifecycle rules for Model Ecosystem.
- Validate inputs, permissions, dependencies, and policy constraints before execution.
- Emit auditable events for lifecycle transitions, failures, recovery, and acceptance.
- Provide deterministic behavior that can be tested and traced across releases.

Functional Requirements

- SHALL expose versioned public interfaces for create, read, update, execute, validate, and audit operations.
- SHALL validate all inbound requests against schema, identity, policy, and dependency rules.
- SHALL support idempotency keys for commands that may be retried.
- SHALL publish events for accepted, rejected, started, completed, failed, recovered, and archived states.
- SHALL produce evidence bundles for review, compliance, and release acceptance.

Non-Functional Requirements

- MUST preserve tenant isolation, least privilege, and traceable authorization.
- MUST support observable latency, throughput, failure rate, queue depth, and recovery metrics.

- MUST recover safely after process restart, duplicate event delivery, or partial storage failure.
- SHOULD degrade gracefully when optional providers, plugins, connectors, or models are unavailable.

Inputs

- User, agent, workflow, runtime, provider, plugin, connector, and governance requests.
- Configuration, policies, feature flags, schemas, secrets references, and dependency metadata.
- Runtime events, task graph state, knowledge records, capability scores, and audit context.

Outputs

- Versioned records, execution results, validation reports, events, metrics, traces, logs, and acceptance evidence.
- Human-readable status summaries for dashboards, operations, release notes, and incident review.

Public Interfaces

- `create_model_ecosystem(request)`
- `get_model_ecosystem_state(id)`
- `validate_model_ecosystem(payload)`
- `execute_model_ecosystem(command)`
- `audit_model_ecosystem(scope)`

Internal Interfaces

- Event Bus for durable event publication and subscription.
- Service Container for dependency injection and lifecycle management.
- Runtime Manager for execution dispatch and cancellation.
- Storage layer for records, snapshots, indexes, and evidence.
- Governance service for approval, policy, risk, and compliance checks.

Events

- `model-ecosystem.created`
- `model-ecosystem.validated`
- `model-ecosystem.started`
- `model-ecosystem.completed`
- `model-ecosystem.failed`
- `model-ecosystem.recovered`
- `model-ecosystem.accepted`

Storage Requirements

- Store canonical records with identifiers, owner, version, state, timestamps, policy version, and trace ID.
- Store immutable event history and mutable read models separately where practical.
- Store evidence, validation reports, and recovery metadata with retention rules.

Security Requirements

- Enforce authentication, authorization, tenant isolation, and field-level redaction.
- Protect secrets by reference only; never persist raw credentials in logs, events, Markdown, or exports.
- Require approval for privileged, destructive, externally visible, financial, or compliance-sensitive actions.

Observability Requirements

- Emit structured logs with correlation IDs and decision reasons.
- Emit metrics for request count, success rate, failure rate, latency, retries, recovery success, and acceptance failures.
- Provide traces across public interface, policy validation, runtime execution, storage, events, and callbacks.

Failure Modes

- Invalid input, missing dependency, expired permission, unavailable runtime, conflicting state, duplicate command, or failed downstream provider.
- Incomplete event delivery, partial persistence, timeout, stalled execution, or acceptance failure.

Recovery Rules

- Retry idempotent commands with bounded backoff.
- Rebuild read models from event history when projections drift.
- Escalate to governance when automatic recovery changes risk, scope, budget, deadline, or external side effects.
- Preserve failed attempts and recovery decisions as audit evidence.

Test Requirements

- Unit tests for schemas, state transitions, permission checks, and event payloads.
- Integration tests for public interfaces, storage, event bus, runtime dependencies, and governance approvals.
- Failure tests for retries, duplicate events, unavailable dependencies, and recovery paths.
- Acceptance tests proving traceability from requirement to evidence.

Acceptance Criteria

- The specification is complete enough to create implementation tickets without hidden architectural decisions.
- Interfaces, events, storage, security, observability, failures, recovery, and tests are documented.
- The document traces to Blueprint, Standards, and release artifacts.

Implementation Notes

- Prefer small versioned interfaces and append-only event history.
- Keep provider-specific details behind adapters.
- Treat auditability and recovery as first-class design constraints.
- Validate schemas before work reaches runtime execution.

Traceability

Source	Relationship
MAL-V06-MODEL-ECOSYSTEM-SPEC-01	Defines v30 implementation requirements for Model Ecosystem.

Source	Relationship
Volume 03	Blueprint source.
Volume 05	Engineering standards source.
RELEASE_NOTES_v30_draft.md	Release evidence.

Version History

Version	Date	Status	Notes
v30 draft	2026-06-29	Draft	Initial specification for Model Ecosystem.

Provider Ecosystem Specification

Title: Provider Ecosystem Specification

Version: v30 draft

Status: Draft

Specification ID: MAL-V06-PROVIDER-ECOSYSTEM-SPEC-02

Related Blueprint Documents

- Volume 03 Master Blueprint
- Volume 04 Canonical Dictionary
- Volume 05 Engineering Standards
- Prior Volume 06 Specifications

Purpose

Define implementation-level requirements for Provider Ecosystem in Mariam AI Enterprise OS so engineering teams can build, test, operate, and govern it consistently.

Scope

- Covers system responsibilities, runtime behavior, interfaces, events, storage, security, observability, and acceptance evidence.
- Includes interactions with Enterprise Core, AI Society, Runtime Ecosystem, Knowledge System, Capability System, Governance, and Infrastructure.
- Excludes application-specific code, vendor credentials, and temporary implementation shortcuts.

Responsibilities

- Maintain authoritative state and lifecycle rules for Provider Ecosystem.
- Validate inputs, permissions, dependencies, and policy constraints before execution.
- Emit auditable events for lifecycle transitions, failures, recovery, and acceptance.
- Provide deterministic behavior that can be tested and traced across releases.

Functional Requirements

- SHALL expose versioned public interfaces for create, read, update, execute, validate, and audit operations.
- SHALL validate all inbound requests against schema, identity, policy, and dependency rules.
- SHALL support idempotency keys for commands that may be retried.
- SHALL publish events for accepted, rejected, started, completed, failed, recovered, and archived states.
- SHALL produce evidence bundles for review, compliance, and release acceptance.

Non-Functional Requirements

- MUST preserve tenant isolation, least privilege, and traceable authorization.
- MUST support observable latency, throughput, failure rate, queue depth, and recovery metrics.

- MUST recover safely after process restart, duplicate event delivery, or partial storage failure.
- SHOULD degrade gracefully when optional providers, plugins, connectors, or models are unavailable.

Inputs

- User, agent, workflow, runtime, provider, plugin, connector, and governance requests.
- Configuration, policies, feature flags, schemas, secrets references, and dependency metadata.
- Runtime events, task graph state, knowledge records, capability scores, and audit context.

Outputs

- Versioned records, execution results, validation reports, events, metrics, traces, logs, and acceptance evidence.
- Human-readable status summaries for dashboards, operations, release notes, and incident review.

Public Interfaces

- `create_provider_ecosystem(request)`
- `get_provider_ecosystem_state(id)`
- `validate_provider_ecosystem(payload)`
- `execute_provider_ecosystem(command)`
- `audit_provider_ecosystem(scope)`

Internal Interfaces

- Event Bus for durable event publication and subscription.
- Service Container for dependency injection and lifecycle management.
- Runtime Manager for execution dispatch and cancellation.
- Storage layer for records, snapshots, indexes, and evidence.
- Governance service for approval, policy, risk, and compliance checks.

Events

- `provider-ecosystem.created`
- `provider-ecosystem.validated`
- `provider-ecosystem.started`
- `provider-ecosystem.completed`
- `provider-ecosystem.failed`
- `provider-ecosystem.recovered`
- `provider-ecosystem.accepted`

Storage Requirements

- Store canonical records with identifiers, owner, version, state, timestamps, policy version, and trace ID.
- Store immutable event history and mutable read models separately where practical.
- Store evidence, validation reports, and recovery metadata with retention rules.

Security Requirements

- Enforce authentication, authorization, tenant isolation, and field-level redaction.
- Protect secrets by reference only; never persist raw credentials in logs, events, Markdown, or exports.
- Require approval for privileged, destructive, externally visible, financial, or compliance-sensitive actions.

Observability Requirements

- Emit structured logs with correlation IDs and decision reasons.
- Emit metrics for request count, success rate, failure rate, latency, retries, recovery success, and acceptance failures.
- Provide traces across public interface, policy validation, runtime execution, storage, events, and callbacks.

Failure Modes

- Invalid input, missing dependency, expired permission, unavailable runtime, conflicting state, duplicate command, or failed downstream provider.
- Incomplete event delivery, partial persistence, timeout, stalled execution, or acceptance failure.

Recovery Rules

- Retry idempotent commands with bounded backoff.
- Rebuild read models from event history when projections drift.
- Escalate to governance when automatic recovery changes risk, scope, budget, deadline, or external side effects.
- Preserve failed attempts and recovery decisions as audit evidence.

Test Requirements

- Unit tests for schemas, state transitions, permission checks, and event payloads.
- Integration tests for public interfaces, storage, event bus, runtime dependencies, and governance approvals.
- Failure tests for retries, duplicate events, unavailable dependencies, and recovery paths.
- Acceptance tests proving traceability from requirement to evidence.

Acceptance Criteria

- The specification is complete enough to create implementation tickets without hidden architectural decisions.
- Interfaces, events, storage, security, observability, failures, recovery, and tests are documented.
- The document traces to Blueprint, Standards, and release artifacts.

Implementation Notes

- Prefer small versioned interfaces and append-only event history.
- Keep provider-specific details behind adapters.
- Treat auditability and recovery as first-class design constraints.
- Validate schemas before work reaches runtime execution.

Traceability

Source	Relationship
MAL-V06-PROVIDER-ECOSYSTEM-SPEC-02	Defines v30 implementation requirements for Provider Ecosystem.

Source	Relationship
Volume 03	Blueprint source.
Volume 05	Engineering standards source.
RELEASE_NOTES_v30_draft.md	Release evidence.

Version History

Version	Date	Status	Notes
v30 draft	2026-06-29	Draft	Initial specification for Provider Ecosystem.

Plugin Runtime Specification

Title: Plugin Runtime Specification

Version: v30 draft

Status: Draft

Specification ID: MAL-V06-PLUGIN-RUNTIME-SPEC-03

Related Blueprint Documents

- Volume 03 Master Blueprint
- Volume 04 Canonical Dictionary
- Volume 05 Engineering Standards
- Prior Volume 06 Specifications

Purpose

Define implementation-level requirements for Plugin Runtime in Mariam AI Enterprise OS so engineering teams can build, test, operate, and govern it consistently.

Scope

- Covers system responsibilities, runtime behavior, interfaces, events, storage, security, observability, and acceptance evidence.
- Includes interactions with Enterprise Core, AI Society, Runtime Ecosystem, Knowledge System, Capability System, Governance, and Infrastructure.
- Excludes application-specific code, vendor credentials, and temporary implementation shortcuts.

Responsibilities

- Maintain authoritative state and lifecycle rules for Plugin Runtime.
- Validate inputs, permissions, dependencies, and policy constraints before execution.
- Emit auditable events for lifecycle transitions, failures, recovery, and acceptance.
- Provide deterministic behavior that can be tested and traced across releases.

Functional Requirements

- SHALL expose versioned public interfaces for create, read, update, execute, validate, and audit operations.
- SHALL validate all inbound requests against schema, identity, policy, and dependency rules.
- SHALL support idempotency keys for commands that may be retried.
- SHALL publish events for accepted, rejected, started, completed, failed, recovered, and archived states.
- SHALL produce evidence bundles for review, compliance, and release acceptance.

Non-Functional Requirements

- MUST preserve tenant isolation, least privilege, and traceable authorization.
- MUST support observable latency, throughput, failure rate, queue depth, and recovery metrics.

- MUST recover safely after process restart, duplicate event delivery, or partial storage failure.
- SHOULD degrade gracefully when optional providers, plugins, connectors, or models are unavailable.

Inputs

- User, agent, workflow, runtime, provider, plugin, connector, and governance requests.
- Configuration, policies, feature flags, schemas, secrets references, and dependency metadata.
- Runtime events, task graph state, knowledge records, capability scores, and audit context.

Outputs

- Versioned records, execution results, validation reports, events, metrics, traces, logs, and acceptance evidence.
- Human-readable status summaries for dashboards, operations, release notes, and incident review.

Public Interfaces

- `create_plugin_runtime(request)`
- `get_plugin_runtime_state(id)`
- `validate_plugin_runtime(payload)`
- `execute_plugin_runtime(command)`
- `audit_plugin_runtime(scope)`

Internal Interfaces

- Event Bus for durable event publication and subscription.
- Service Container for dependency injection and lifecycle management.
- Runtime Manager for execution dispatch and cancellation.
- Storage layer for records, snapshots, indexes, and evidence.
- Governance service for approval, policy, risk, and compliance checks.

Events

- `plugin-runtime.created`
- `plugin-runtime.validated`
- `plugin-runtime.started`
- `plugin-runtime.completed`
- `plugin-runtime.failed`
- `plugin-runtime.recovered`
- `plugin-runtime.accepted`

Storage Requirements

- Store canonical records with identifiers, owner, version, state, timestamps, policy version, and trace ID.
- Store immutable event history and mutable read models separately where practical.
- Store evidence, validation reports, and recovery metadata with retention rules.

Security Requirements

- Enforce authentication, authorization, tenant isolation, and field-level redaction.
- Protect secrets by reference only; never persist raw credentials in logs, events, Markdown, or exports.
- Require approval for privileged, destructive, externally visible, financial, or compliance-sensitive actions.

Observability Requirements

- Emit structured logs with correlation IDs and decision reasons.
- Emit metrics for request count, success rate, failure rate, latency, retries, recovery success, and acceptance failures.
- Provide traces across public interface, policy validation, runtime execution, storage, events, and callbacks.

Failure Modes

- Invalid input, missing dependency, expired permission, unavailable runtime, conflicting state, duplicate command, or failed downstream provider.
- Incomplete event delivery, partial persistence, timeout, stalled execution, or acceptance failure.

Recovery Rules

- Retry idempotent commands with bounded backoff.
- Rebuild read models from event history when projections drift.
- Escalate to governance when automatic recovery changes risk, scope, budget, deadline, or external side effects.
- Preserve failed attempts and recovery decisions as audit evidence.

Test Requirements

- Unit tests for schemas, state transitions, permission checks, and event payloads.
- Integration tests for public interfaces, storage, event bus, runtime dependencies, and governance approvals.
- Failure tests for retries, duplicate events, unavailable dependencies, and recovery paths.
- Acceptance tests proving traceability from requirement to evidence.

Acceptance Criteria

- The specification is complete enough to create implementation tickets without hidden architectural decisions.
- Interfaces, events, storage, security, observability, failures, recovery, and tests are documented.
- The document traces to Blueprint, Standards, and release artifacts.

Implementation Notes

- Prefer small versioned interfaces and append-only event history.
- Keep provider-specific details behind adapters.
- Treat auditability and recovery as first-class design constraints.
- Validate schemas before work reaches runtime execution.

Traceability

Source	Relationship
MAL-V06-PLUGIN-RUNTIME-SPEC-03	Defines v30 implementation requirements for Plugin Runtime.

Source	Relationship
Volume 03	Blueprint source.
Volume 05	Engineering standards source.
RELEASE_NOTES_v30_draft.md	Release evidence.

Version History

Version	Date	Status	Notes
v30 draft	2026-06-29	Draft	Initial specification for Plugin Runtime.

Connector Runtime Specification

Title: Connector Runtime Specification

Version: v30 draft

Status: Draft

Specification ID: MAL-V06-CONNECTOR-RUNTIME-SPEC-04

Related Blueprint Documents

- Volume 03 Master Blueprint
- Volume 04 Canonical Dictionary
- Volume 05 Engineering Standards
- Prior Volume 06 Specifications

Purpose

Define implementation-level requirements for Connector Runtime in Mariam AI Enterprise OS so engineering teams can build, test, operate, and govern it consistently.

Scope

- Covers system responsibilities, runtime behavior, interfaces, events, storage, security, observability, and acceptance evidence.
- Includes interactions with Enterprise Core, AI Society, Runtime Ecosystem, Knowledge System, Capability System, Governance, and Infrastructure.
- Excludes application-specific code, vendor credentials, and temporary implementation shortcuts.

Responsibilities

- Maintain authoritative state and lifecycle rules for Connector Runtime.
- Validate inputs, permissions, dependencies, and policy constraints before execution.
- Emit auditable events for lifecycle transitions, failures, recovery, and acceptance.
- Provide deterministic behavior that can be tested and traced across releases.

Functional Requirements

- SHALL expose versioned public interfaces for create, read, update, execute, validate, and audit operations.
- SHALL validate all inbound requests against schema, identity, policy, and dependency rules.
- SHALL support idempotency keys for commands that may be retried.
- SHALL publish events for accepted, rejected, started, completed, failed, recovered, and archived states.
- SHALL produce evidence bundles for review, compliance, and release acceptance.

Non-Functional Requirements

- MUST preserve tenant isolation, least privilege, and traceable authorization.
- MUST support observable latency, throughput, failure rate, queue depth, and recovery metrics.

- MUST recover safely after process restart, duplicate event delivery, or partial storage failure.
- SHOULD degrade gracefully when optional providers, plugins, connectors, or models are unavailable.

Inputs

- User, agent, workflow, runtime, provider, plugin, connector, and governance requests.
- Configuration, policies, feature flags, schemas, secrets references, and dependency metadata.
- Runtime events, task graph state, knowledge records, capability scores, and audit context.

Outputs

- Versioned records, execution results, validation reports, events, metrics, traces, logs, and acceptance evidence.
- Human-readable status summaries for dashboards, operations, release notes, and incident review.

Public Interfaces

- `create_connector_runtime(request)`
- `get_connector_runtime_state(id)`
- `validate_connector_runtime(payload)`
- `execute_connector_runtime(command)`
- `audit_connector_runtime(scope)`

Internal Interfaces

- Event Bus for durable event publication and subscription.
- Service Container for dependency injection and lifecycle management.
- Runtime Manager for execution dispatch and cancellation.
- Storage layer for records, snapshots, indexes, and evidence.
- Governance service for approval, policy, risk, and compliance checks.

Events

- `connector-runtime.created`
- `connector-runtime.validated`
- `connector-runtime.started`
- `connector-runtime.completed`
- `connector-runtime.failed`
- `connector-runtime.recovered`
- `connector-runtime.accepted`

Storage Requirements

- Store canonical records with identifiers, owner, version, state, timestamps, policy version, and trace ID.
- Store immutable event history and mutable read models separately where practical.
- Store evidence, validation reports, and recovery metadata with retention rules.

Security Requirements

- Enforce authentication, authorization, tenant isolation, and field-level redaction.
- Protect secrets by reference only; never persist raw credentials in logs, events, Markdown, or exports.
- Require approval for privileged, destructive, externally visible, financial, or compliance-sensitive actions.

Observability Requirements

- Emit structured logs with correlation IDs and decision reasons.
- Emit metrics for request count, success rate, failure rate, latency, retries, recovery success, and acceptance failures.
- Provide traces across public interface, policy validation, runtime execution, storage, events, and callbacks.

Failure Modes

- Invalid input, missing dependency, expired permission, unavailable runtime, conflicting state, duplicate command, or failed downstream provider.
- Incomplete event delivery, partial persistence, timeout, stalled execution, or acceptance failure.

Recovery Rules

- Retry idempotent commands with bounded backoff.
- Rebuild read models from event history when projections drift.
- Escalate to governance when automatic recovery changes risk, scope, budget, deadline, or external side effects.
- Preserve failed attempts and recovery decisions as audit evidence.

Test Requirements

- Unit tests for schemas, state transitions, permission checks, and event payloads.
- Integration tests for public interfaces, storage, event bus, runtime dependencies, and governance approvals.
- Failure tests for retries, duplicate events, unavailable dependencies, and recovery paths.
- Acceptance tests proving traceability from requirement to evidence.

Acceptance Criteria

- The specification is complete enough to create implementation tickets without hidden architectural decisions.
- Interfaces, events, storage, security, observability, failures, recovery, and tests are documented.
- The document traces to Blueprint, Standards, and release artifacts.

Implementation Notes

- Prefer small versioned interfaces and append-only event history.
- Keep provider-specific details behind adapters.
- Treat auditability and recovery as first-class design constraints.
- Validate schemas before work reaches runtime execution.

Traceability

Source	Relationship
MAL-V06-CONNECTOR-RUNTIME-SPEC-04	Defines v30 implementation requirements for Connector Runtime.

Source	Relationship
Volume 03	Blueprint source.
Volume 05	Engineering standards source.
RELEASE_NOTES_v30_draft.md	Release evidence.

Version History

Version	Date	Status	Notes
v30 draft	2026-06-29	Draft	Initial specification for Connector Runtime.